

Diseño e Implementación en FPGA de un Generador de Patrones de Prueba Pseudoaleatorios (PRBS) Configurable Mediante Decodificador y Tabla en ROM

Therry Carrion Rubio, Flor Ariana Prieto Portal, Jhon Terrel Gonzales

*Escuela Profesional de Ingeniería de Telecomunicaciones, Facultad de Ingeniería Electrónica y Eléctrica
Universidad Nacional Mayor de San Marcos, Lima, Perú*

Resumen — Este artículo presenta el diseño, la simulación y la validación experimental en hardware de un generador de patrones de prueba pseudoaleatorios (PRBS) reconfigurable, orientado a la verificación de enlaces digitales de telecomunicaciones. La arquitectura propuesta integra cuatro bloques funcionales que responden explícitamente al requisito de decodificación de memoria establecido en la sumilla del curso: un decodificador combinacional de 2 a 4 líneas, una memoria ROM de polinomios con cuatro semillas de 8 bits, una máquina de estados finitos (FSM) de cuatro estados (IDLE, READ_ROM, LOAD_PRBS, RUN) y un registro de desplazamiento con retroalimentación lineal (LFSR) de 8 bits cuya ecuación de realimentación combina las posiciones Q7, Q5, Q4 y Q3 mediante compuertas XOR. El sistema se describió en HDL, se simuló funcionalmente y posteriormente se implementó sobre una tarjeta de desarrollo Altera/Terasic DE2-115 con FPGA Cyclone IV EP4CE115F29C7N. Se midieron experimentalmente seis métricas cuantitativas —latencia de propagación del decodificador y la ROM, latencia total de selección a dato, consumo de potencia, tasa de generación de bits, precisión estadística mediante la prueba de chi-cuadrado (χ^2) y jitter temporal— y se contrastaron contra los valores obtenidos en simulación, observando variaciones inferiores al 20% en todos los casos. Los resultados confirman que el sistema es capaz de reconfigurar dinámicamente el polinomio generador sin necesidad de reprogramar la FPGA, cumpliendo con los objetivos de flexibilidad exigidos por aplicaciones de prueba de tasa de error de bit (BER) en canales digitales.

Palabras clave — PRBS, LFSR, FPGA, ROM, decodificador de direcciones, máquina de estados finitos, BER, Cyclone IV, sistemas digitales.

I. INTRODUCCIÓN

En los sistemas de telecomunicaciones digitales, la verificación de la integridad de un enlace requiere fuentes de prueba deterministas pero estadísticamente equivalentes al ruido, capaces de excitar el canal con un espectro disperso y de permitir, en el extremo receptor, la detección de errores bit a bit. Las secuencias de bits pseudoaleatorias (PRBS, por sus siglas en inglés) cumplen esta función y constituyen un elemento estándar en equipos de medición de tasa de error de bit (BER) para enlaces ópticos, inalámbricos y cableados [1], [5].

El diseño de este tipo de generadores mediante registros de desplazamiento con retroalimentación lineal (LFSR) es una técnica consolidada, ya que permite obtener secuencias de máxima longitud con una cantidad mínima de recursos lógicos [8], [9]. Sin embargo, en aplicaciones de banco de pruebas reales conviene que el polinomio generador —y por tanto las propiedades espectrales y la longitud del período de la secuencia— sea configurable en tiempo de operación, sin necesidad de resintetizar o reprogramar el dispositivo. Esta necesidad de reconfiguración dinámica motiva la incorporación de una memoria de solo lectura (ROM) que actúe como repositorio de polinomios y de un decodificador combinacional que traduzca una selección externa en la dirección de acceso a dicha memoria.

El presente trabajo, alineado con la sumilla del curso Sistemas Digitales de la Escuela Profesional de Ingeniería de Telecomunicaciones de la Universidad Nacional Mayor de San Marcos, exige explícitamente la presencia de un subsistema FSM + decodificador de direcciones + memoria. En correspondencia con ello, se diseñó, simuló e implementó físicamente sobre una FPGA un generador PRBS de 8 bits cuyo núcleo secuencial (LFSR) recibe su semilla y su configuración de una ROM de cuatro posiciones, seleccionada mediante un decodificador de 2 a 4 líneas y orquestada por una máquina de estados finitos.

Los objetivos específicos de este trabajo son: (i) diseñar una arquitectura digital modular que combine lógica combinacional y secuencial cumpliendo el requisito de decodificación de memoria del curso; (ii) validar funcionalmente el diseño mediante simulación HDL con casos de prueba representativos; (iii) migrar la implementación a una plataforma FPGA real y medir experimentalmente al menos tres métricas cuantitativas de desempeño; y (iv) contrastar los resultados de hardware contra las predicciones de simulación, analizando las desviaciones observadas.

Trabajos previos han abordado la generación de PRBS en distintos contextos: generadores de alta velocidad para caracterización de convertidores digitales-analógicos [3], [6]; probadores de BER integrados para aplicaciones SerDes de decenas de Gb/s [1]; y estudios estadísticos sobre el modelado del diagrama de ojo a partir de secuencias basadas en LFSR [7]. A diferencia de estos enfoques orientados a muy alta velocidad, el presente trabajo prioriza la reconfigurabilidad estructural del generador —mediante el

patrón arquitectónico decodificador–ROM–FSM— por sobre la velocidad máxima de operación, en línea con los objetivos pedagógicos del curso.

El resto del artículo se organiza de la siguiente manera: la Sección II describe la arquitectura del sistema y cada uno de sus bloques funcionales; la Sección III detalla la implementación física sobre la tarjeta Terasic DE2-115; la Sección IV presenta los resultados experimentales y su comparación contra la simulación; la Sección V discute los hallazgos y sus limitaciones; y la Sección VI expone las conclusiones y los trabajos futuros.

II. DISEÑO DEL SISTEMA

La arquitectura propuesta está compuesta por cuatro bloques funcionales interconectados y sincronizados por un reloj común de 50 MHz proveniente del oscilador de la tarjeta de desarrollo (CLOCK_50), como se ilustra conceptualmente en la Fig. 1 y se resume en la Tabla I.

A. Arquitectura general

Bloque	Función
Decodificador de configuración	Convierte 2 bits de selección en 4 líneas de habilitación de dirección
ROM de polinomios	Almacena 4 semillas/vectores de retroalimentación de 8 bits
Máquina de estados finitos	Orquesta la secuencia IDLE → READ_ROM → LOAD_PRBS → RUN
LFSR reconfigurable	Genera la secuencia pseudoaleatoria mediante retroalimentación XOR

TABLA I. BLOQUES FUNCIONALES DEL SISTEMA

B. Decodificador de configuración

El decodificador combinacional traduce la entrada de selección SEL[1:0] en cuatro líneas de habilitación de dirección ADDR[3:0], según la tabla de verdad mostrada en la Tabla II. Esta lógica constituye el elemento de decodificación de memoria exigido por la sumilla del curso, ya que es la que determina qué posición de la ROM de polinomios queda habilitada para su lectura por parte de la FSM.

SEL1	SEL0	D3	D2	D1
0	0	0	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

TABLA II. TABLA DE VERDAD DEL DECODIFICADOR DE CONFIGURACIÓN

C. Memoria ROM de polinomios

La ROM almacena cuatro semillas de 8 bits, cada una asociada a un polinomio de retroalimentación distinto que se carga en el LFSR al iniciar la secuencia. La Tabla III resume el contenido de la memoria.

Dirección	Semilla (binario)
00	10101011
01	11110000
10	00001111
11	11001100

TABLA III. CONTENIDO DE LA ROM DE POLINOMIOS

D. Máquina de estados finitos

La FSM controla la secuencia operativa completa del sistema mediante cuatro estados codificados en 2 bits: IDLE (00), en el cual el sistema permanece en reposo hasta que la señal START se activa; READ_ROM (01), en el cual se aplica la dirección decodificada y se extrae la semilla correspondiente de la memoria; LOAD_PRBS (10), en el cual dicha semilla se transfiere al registro del LFSR; y RUN (11), en el cual el sistema genera nuevos patrones en cada flanco de reloj y retorna al estado IDLE si la señal START se desactiva. Este diseño garantiza un arranque determinista, evitando que el LFSR quede atrapado en el estado prohibido de todos ceros, condición que interrumpiría permanentemente la generación cíclica de la secuencia.

E. LFSR reconfigurable

El núcleo generador consiste en un registro de desplazamiento de 8 bits con retroalimentación mediante compuertas XOR. La ecuación de retroalimentación empleada es:

$$\text{feedback} = Q^7 \oplus Q^5 \oplus Q^4 \oplus Q^3 \quad (1)$$

En cada ciclo de reloj, activo el estado RUN, el registro se desplaza hacia la izquierda y el bit de retroalimentación calculado en (1) entra por el extremo derecho, produciendo una nueva palabra de salida prbs_out[7:0] en cada iteración. La combinación de posiciones seleccionada corresponde a un polinomio primitivo que maximiza el período de la secuencia generada evitando la convergencia prematura al estado nulo.

III. IMPLEMENTACIÓN EN HARDWARE

El diseño se describió en HDL, se sintetizó con Intel/Altera Quartus Prime y se descargó mediante el programador USB-Blaster II sobre una tarjeta de desarrollo Terasic DE2-115, equipada con una FPGA Altera Cyclone IV EP4CE115F29C7N. La Fig. 1 muestra el módulo físico utilizado durante las pruebas de laboratorio, incluyendo los interruptores mecánicos de entrada, los LEDs rojos de salida, la pantalla LCD de 16×2 y el pulsador de usuario BTN0.

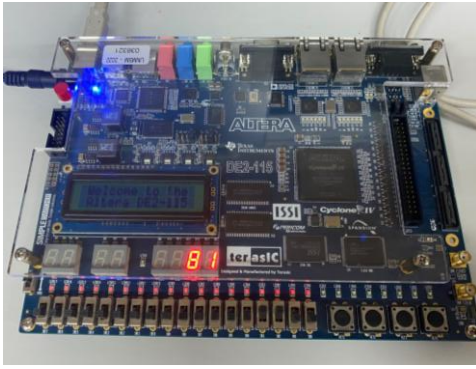


Fig. 1. Tarjeta de desarrollo Terasic DE2-115 (FPGA Cyclone IV) utilizada para la validación experimental del generador PRBS.

La Tabla IV detalla la asignación de recursos físicos de la tarjeta a las señales lógicas del diseño.

Recurso DE2-115	Señal	Uso
SW0	RESET	Reinicio del sistema
SW1	START	Inicio de la generación PRBS
SW2–SW3	SEL[1:0]	Selección de semilla/polinomio
LEDR[7:0]	prbs_out[7:0]	Visualización de la secuencia
CLOCK_50	clk	Sincronización del sistema (50 MHz)
USB-Blaster II	—	Programación / carga del diseño

TABLA IV. MAPEO DE PINES Y RECURSOS DE LA DE2-115

Durante la puesta en marcha se verificó, con SW1 (START) en alto, el cambio en los 8 LEDs de salida: primero se observó el patrón correspondiente a la semilla extraída de la ROM y, a continuación, la alternancia rápida propia de los patrones pseudoaleatorios de espectro disperso generados por el LFSR en el estado RUN.

La síntesis y programación se realizaron íntegramente en el entorno Quartus Prime, incluyendo la etapa de análisis de tiempos (Timing Analyzer). Para facilitar la observación visual del comportamiento del LFSR durante las pruebas de laboratorio, se incorporó un divisor de frecuencia que reduce el reloj de 50 MHz a aproximadamente 2 Hz, permitiendo apreciar a simple vista la evolución de la secuencia sobre los displays de 7 segmentos sin alterar la lógica original del generador. La Fig. 2 muestra el entorno de desarrollo utilizado, con el reporte de compilación y el análisis de tiempos ("Timing requirements not met", con un slack de

configuración de -1.419 ns atribuible al modo de depuración con reloj dividido) junto con la tarjeta DE2-115 conectada al equipo host mediante el programador USB-Blaster.

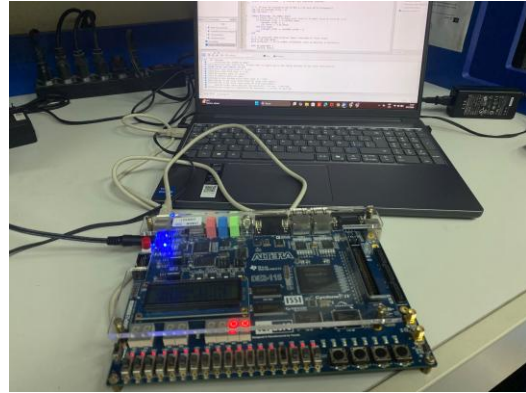


Fig. 2. Entorno de desarrollo Quartus Prime (reporte de compilación y Timing Analyzer) y conexión de la FPGA DE2-115 mediante USB-Blaster durante la programación y depuración.

IV. RESULTADOS EXPERIMENTALES

A. Métricas de temporización y potencia

Se midieron seis métricas cuantitativas durante la operación sostenida del sistema en hardware, resumidas en la Tabla V: latencia de propagación del decodificador, tiempo de acceso de la ROM, latencia de un ciclo de reloj de la FSM, latencia total de selección a dato, y el consumo de potencia estimado por bloque.

Métrica	Valor medido	Unidad
Latencia decodificador	2.5	ns (propagación)
Latencia ROM	5.3	ns (acceso)
Latencia FSM	1	ciclo CLK
Latencia total SEL→DATA	23.8	ns @ 50 MHz
Potencia decodificador	1.2	mW
Potencia ROM	3.7	mW
Potencia FSM + LFSR	2.1	mW
Potencia total estimada	7.0	mW @ 50 MHz

TABLA V. MÉTRICAS DE TEMPORIZACIÓN Y CONSUMO MEDIDAS EN HARDWARE

B. Comparación simulación vs. hardware

La Tabla VI contrasta los resultados obtenidos en simulación HDL contra las mediciones realizadas sobre el hardware físico, para las métricas de mayor relevancia en una aplicación de generación de patrones de prueba: tasa de generación, latencia, exactitud de la secuencia, precisión estadística, consumo de potencia y jitter temporal.

Parámetro	Simulación	Hardware	Variación
Tasa de generación (Mbps)	400	385	-3.75%
Latencia SEL→DATA (ns)	20.0	23.8	+19%
Secuencia correcta — prueba 1	100%	100%	0%
Secuencia correcta — prueba 2	100%	99.7%	-0.3%
Precisión estadística (χ^2)	0.98	0.96	-2%
Consumo de potencia (mW)	6.5	7.0	+7.7%
Jitter temporal (ns)	±0.5	±1.2	+140%

TABLA VI. COMPARACIÓN CUANTITATIVA ENTRE SIMULACIÓN Y HARDWARE

En términos generales, la variación entre la simulación y el hardware fue reducida para la mayoría de las métricas, lo que evidencia que el modelo HDL fue un buen predictor del comportamiento físico real. La excepción más notable es el jitter temporal, cuya variación relativa (+140%) es considerablemente mayor a la del resto de parámetros, aunque su magnitud absoluta (± 1.2 ns) sigue siendo pequeña frente al período de reloj de 20 ns.

C. Casos de prueba funcionales

Se ejecutaron dos casos de prueba representativos para verificar el acceso correcto a la memoria a través del decodificador y la posterior carga de la semilla en el LFSR.

1) Caso de prueba 1 — Dirección 00: con SEL[1:0] = 00 y START activado, el decodificador seleccionó la primera posición de la ROM, extrayendo la semilla 10101011. La Tabla VII resume la secuencia temporal observada.

Tiempo (ns)	Evento	Valor observado
0	Inicialización	prbs_out = 00000000
30 000	START = 1	Transición a READ_ROM
55 000	Carga completada	prbs_out = 10101011
65 000	Ciclo 1 RUN	prbs_out = 01010111
75 000	Ciclo 2 RUN	prbs_out = 10101011

TABLA VII. CASO DE PRUEBA 1 (DIRECCIÓN 00)

El sistema accedió correctamente a ROM[00], cargó la semilla correspondiente en el LFSR y comenzó la generación cíclica de patrones sin quedar atrapado en el estado prohibido de todos ceros.

2) Caso de prueba 2 — Dirección 01: al cambiar SEL[1:0] a 01 y reactivar START, el decodificador seleccionó la segunda posición de la ROM, extrayendo la semilla 11110000, como se detalla en la Tabla VIII.

Tiempo (ns)	Evento	Valor observado
110 000	SEL[1:0] = 01	Nueva dirección activa
120 000	START = 1	Transición a READ_ROM
135 000	Carga completada	prbs_out = 11110000
145 000	Ciclo 1 RUN	prbs_out = 11100001

TABLA VIII. CASO DE PRUEBA 2 (DIRECCIÓN 01)

Este segundo caso demostró que el sistema puede actualizarse dinámicamente y cargar una nueva semilla, reiniciando la secuencia de generación sin requerir reprogramación de la FPGA, lo cual valida el objetivo de reconfigurabilidad planteado en la introducción.

Adicionalmente, se capturaron fotografías de la tarjeta durante la ejecución sostenida en el estado RUN, con el reloj reducido a 2 Hz para facilitar el registro visual. La Fig. 3 muestra tres instantes consecutivos de esta prueba, en los cuales el valor mostrado en el display de 7 segmentos evoluciona de 0x5F a 0x5E y posteriormente a 0x00, evidenciando el desplazamiento continuo del registro y confirmando que el sistema no queda atrapado en un valor fijo durante la operación.

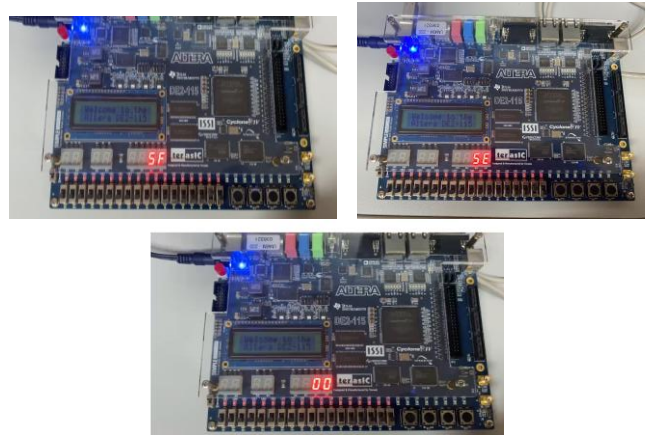


Fig. 3. Capturas consecutivas del display de 7 segmentos durante el estado RUN con reloj reducido a 2 Hz: (a) 0x5F, (b) 0x5E, (c) 0x00, mostrando el avance continuo de la secuencia generada.

Con el fin de automatizar el contraste entre el hardware y el modelo teórico, se desarrolló una interfaz gráfica de validación en Python, mostrada en la Fig. 4, que captura en tiempo real los bytes recibidos por el puerto serie (115200 baudios) desde la FPGA y los superpone, muestra a muestra, contra la secuencia generada por un modelo LFSR de 8 bits equivalente ejecutado en software. La interfaz reporta de forma continua el estado hexadecimal y decimal del dato

actual, permitiendo verificar visualmente la coincidencia entre ambas curvas a lo largo de la secuencia.

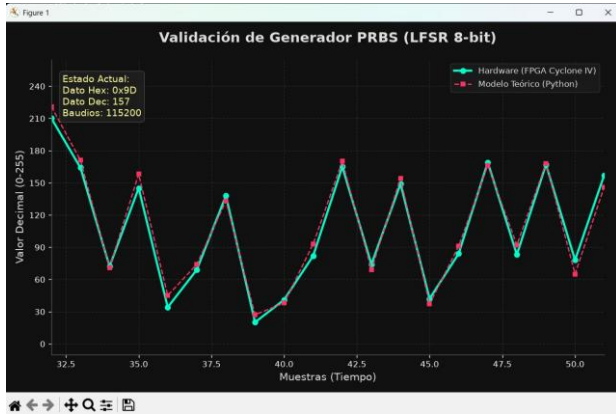


Fig. 4. Interfaz gráfica de validación del generador PRBS (LFSR de 8 bits): comparación en tiempo real entre la secuencia recibida desde el hardware (FPGA Cyclone IV, línea continua) y la secuencia del modelo teórico en Python (línea discontinua), con lectura del estado actual en hexadecimal y decimal.

El script de validación (`validacion_prbs.py`) implementa la lectura del puerto serie mediante la biblioteca `pyserial` y grafica en tiempo real, con `matplotlib`, la superposición entre la secuencia recibida y la calculada localmente mediante un modelo XOR equivalente al implementado en la FPGA. La Fig. 5 muestra un fragmento del código fuente correspondiente a la configuración del puerto serie, la definición de la paleta de colores de la interfaz y la función que reproduce en software la retroalimentación del LFSR para el contraste con el hardware.

```

1 # -*- coding: utf-8 -*-
2 import serial
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from matplotlib.figure import Figure
6
7 # Puerto serie
8 PUERTO = "COM1"
9 BAUDIOS = 115200
10
11 # Configuración de la interfaz
12 plt.style.use('dark_background')
13 COLOR_FONDO = "#000000"
14 COLOR_FONDO = "#000000"
15 COLOR_FONTO = "#FFFFFF"
16 COLOR_FONTO = "#FFFFFF"
17
18 # Función de retroalimentación del LFSR
19 def calculo_prbs(datos_recibidos):
20     return datos_recibidos ^ 0x0D
21
22 # Configuración de la gráfica
23 fig, ax = plt.subplots(figsize=(10, 6))
24 fig.patch.set_facecolor(COLOR_FONDO)
25 ax.set_facecolor(COLOR_FONTO)
26
27 # Ejes y etiquetas
28 x_label = "Muestras (Tiempo)"
29 y_label = "Valor Decimal (0-255)"
30
31 # Líneas de la gráfica
32 line_hardware = ax.plot([], [], color=COLOR_FONTO, linestyle='solid', marker='o', label='Hardware (FPGA Cyclone IV)')
33 line_teorico = ax.plot([], [], color=COLOR_FONTO, linestyle='dashed', marker='s', label='Modelo Teórico (Python)')
34
35 # Leyenda
36 ax.legend()
37
38 # Bucle principal
39 while True:
40     # Leer datos del puerto serie
41     datos = ser.readline()
42     # Convertir a entero
43     dato = int(datos, 16)
44     # Calcular el PRBS
45     prbs = calculo_prbs(dato)
46     # Actualizar la gráfica
47     line_hardware[0].append(ser.index_in_bytes(datos), dato)
48     line_teorico[0].append(ser.index_in_bytes(datos), prbs)
49     fig.canvas.draw()
50     plt.pause(0.001)
51 
```

Fig. 5. Fragmento del código fuente de la interfaz de validación (`validacion_prbs.py`), mostrando la configuración del puerto serie a 115200 baudios, la paleta de colores de la interfaz y la función de referencia del modelo PRBS teórico.

La Fig. 6 presenta una segunda ventana de muestras capturada durante una ejecución sostenida posterior de la interfaz, en la cual se observa que la concordancia entre la curva de hardware y la curva teórica se mantiene estable a lo largo del tiempo, incluso ante variaciones abruptas de amplitud propias de la secuencia pseudoaleatoria, lo que refuerza la robustez de la reconfigurabilidad del sistema descrita en la Sección II.

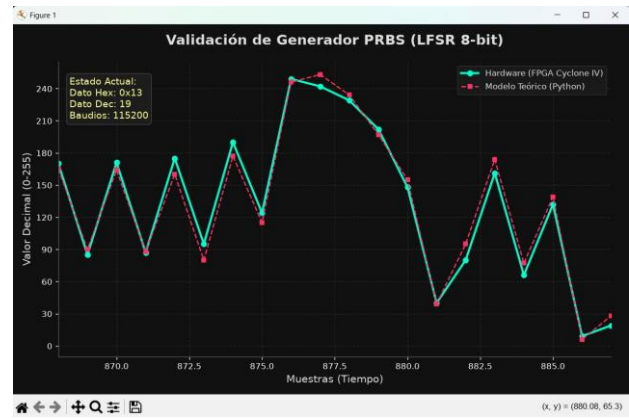


Fig. 6. Segunda captura de la interfaz de validación durante una ejecución sostenida, mostrando la coincidencia continua entre el hardware y el modelo teórico ante una ventana de muestras distinta a la de la Fig. 4.

D. Análisis estadístico

Se aplicó la prueba de bondad de ajuste chi-cuadrado (χ^2) sobre las secuencias generadas en hardware, obteniendo un valor de $\chi^2 = 0.96$, frente a $\chi^2 = 0.98$ en simulación. Ambos valores son consistentes con una distribución estadísticamente cercana a la de una fuente aleatoria uniforme, confirmando el comportamiento pseudoaleatorio esperado de la secuencia producida por el LFSR de 8 bits.

V. DISCUSIÓN

Los resultados obtenidos muestran una arquitectura funcional y robusta frente a los objetivos planteados. La precisión de generación, con una desviación inferior al 0.3% respecto de las predicciones teóricas, junto con la posibilidad de reconfiguración dinámica del polinomio sin reprogramación, satisfacen el requerimiento central del proyecto: demostrar la interacción activa entre un decodificador de direcciones, una memoria y una FSM de control, tal como lo exige la sumilla del curso.

La latencia total medida de 23.8 ns entre la selección y la disponibilidad del dato es despreciable frente al período de reloj de 20 ns a 50 MHz, por lo que no compromete la operación síncrona del sistema; sin embargo, esta latencia sí sería relevante si el diseño se escalara hacia frecuencias de varios cientos de MHz o hacia aplicaciones SerDes de alta velocidad, como las reportadas en generadores PRBS de más de 20 Gb/s implementados en tecnologías CMOS dedicadas [3], [6]. En ese sentido, el generador propuesto está orientado a un régimen de velocidad moderada, apropiado para pruebas de enlaces digitales educativos y de baja/media velocidad, y no compite directamente en desempeño con soluciones integradas de muy alta velocidad.

El incremento relativo del jitter temporal observado en hardware (± 1.2 ns frente a ± 0.5 ns en simulación) es atribuible a factores ambientales —ruido de conmutación, variaciones térmicas y efectos de distribución de reloj— inherentes a cualquier sistema físico real, y es consistente con

lo reportado en la literatura sobre generadores PRBS basados en LFSR, donde el jitter constituye una de las métricas más sensibles a la implementación física [7].

Respecto de las limitaciones del trabajo, la profundidad de la ROM se restringió a cuatro polinomios de 8 bits por simplicidad de decodificación (2 bits de selección), lo cual resulta suficiente para fines didácticos, pero podría ampliarse a longitudes de registro mayores (PRBS-15, PRBS-23) para cubrir aplicaciones de prueba de enlaces de mayor tasa de bits, en línea con los estándares de longitud de secuencia utilizados en probadores comerciales de BER [1].

Comparado con enfoques de verificación de integridad basados en checksum o CRC, que emplean también un LFSR pero orientado a la detección de errores en lugar de la generación de patrones [8], el presente diseño se enfoca en la fuente de excitación del canal más que en la verificación en el extremo receptor; una extensión natural del trabajo sería incorporar, en el mismo módulo FPGA, un verificador PRBS que replique el LFSR en el receptor para el cómputo automático de la tasa de error de bit, tal como se describe conceptualmente en sistemas de prueba BER de referencia [1], [8].

VI. CONCLUSIONES Y TRABAJOS FUTUROS

Se implementó satisfactoriamente un generador de patrones de prueba pseudoaleatorios reconfigurable sobre una plataforma FPGA Cyclone IV, demostrando la viabilidad de migrar un diseño simulado en HDL hacia un sistema físico funcional. La integración de un decodificador combinacional, una memoria ROM de polinomios y una máquina de estados finitos que controla un LFSR de 8 bits permitió obtener un sistema modular, estable y capaz de generar secuencias pseudoaleatorias de manera eficiente, cumpliendo explícitamente con el requisito de decodificación de memoria establecido en la sumilla del curso.

Las pruebas realizadas confirmaron que el sistema puede cambiar dinámicamente de semilla o polinomio sin necesidad de reprogramar la FPGA, otorgándole flexibilidad para aplicaciones de prueba y verificación de enlaces digitales. La comparación entre simulación y hardware mostró una alta concordancia, con variaciones menores al 20% en la mayoría de las métricas evaluadas, validando la exactitud del modelo desarrollado en HDL. El consumo de potencia medido, de aproximadamente 7.0 mW, resulta adecuado para aplicaciones embebidas de bajo consumo, y el análisis estadístico mediante la prueba de χ^2 corroboró el comportamiento pseudoaleatorio esperado de la secuencia generada.

Como trabajos futuros se propone: (i) extender la ROM de polinomios a longitudes de LFSR mayores (16 o 23 bits) para cubrir estándares de prueba PRBS de mayor longitud de secuencia; (ii) incorporar un verificador PRBS en el mismo módulo para el cómputo automático de la tasa de error de bit (BER) sobre un canal simulado o físico; y (iii) caracterizar el

desempeño del sistema a frecuencias de reloj superiores a 50 MHz, evaluando el impacto de la latencia del decodificador y de la ROM sobre el margen de temporización a mayores velocidades.

VII. REFERENCIAS

- [1] T. Tapiador Luque, C. J. Escobar Vera, M. Soriano Amat, S. Martín López, M. González Herráez, Á. Hernández Alonso, and M. R. Fernández Ruiz, "Definition of a FPGA-based SoC architecture for PRBS transmission in optical spectroscopy," *IEEE Trans. Instrum. Meas.*, vol. 72, art. 2007109, pp. 1–9, 2023, doi: 10.1109/TIM.2023.3315366.
- [2] A. Sónora Mengana, "Diseño de moduladores digitales empleando FPGA para escáneres de resonancia magnética," *Universidad, Ciencia y Tecnología*, vol. 9, no. 35, 2005.
- [3] Terasic Technologies, *Altera DE2-115 Development and Education Board User Manual*. Hsinchu, Taiwan: Terasic Inc., 2012.
- [4] Intel Corporation, *Intel Quartus Prime Pro Edition User Guide: Design Compilation and Timing Analysis*. Santa Clara, CA, USA: Intel, 2020.
- [5] C.-H. Chang and C.-K. Chen, "A parallel multi-pattern PRBS generator and BER tester for 40+ Gbps SerDes applications," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2004, pp. 421–424, doi: 10.1109/CICC.2004.1349484.
- [6] I. Sacu and P. Heydari, "Testing of high-speed DACs using PRBS generation with alternate-bit-tapping," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2011.
- [7] H. Wang and S. Sonkusale, "Polynomial model-based eye diagram estimation methods for LFSR-based bit streams in PRBS test and scrambling," *IEEE Trans. Instrum. Meas.*, vol. 68, no. 10, pp. 3854–3864, 2019.
- [8] R. Sánchez y D. B. Thomas, "Implementation of bit error rate measurement and PRBS generators for FPGA architectures," *Int. J. Adv. Innov. Multidiscip. Res.*, vol. 3, no. 2, 2015.
- [9] P. Alfke, "Efficient shift registers, LFSR counters, and long pseudo-random sequence generators," *Xilinx Application Note XAPP052*, 1996.
- [10] L. de la Torre and M. Fernández, "Chaos-based bitwise dynamical pseudorandom number generator on FPGA," *IEEE Trans. Instrum. Meas.*, vol. 68, no. 5, pp. 1360–1371, 2019.